

Perennial — Product Design Document

Version: 0.1 (hackathon MVP) · **Date:** 2026-09-25 **One-liner:** A gardening agent with explicit, mutable memory — it keeps a living plan across seasons instead of drowning in history. **Tagline:** *The garden that remembers.*

1. Problem

Long-horizon agents degrade: observations accumulate, context grows, cost and latency climb, reliability falls. The standard fix — ever-longer transcripts — is the disease.

Gardeners live the human version: a garden is a multi-year project with dozens of concurrent threads (plantings, weather, pests, soil), and its "memory" is a notebook plus seed-packet scribbles. Two seasons later, nobody remembers why the carrots failed.

2. Users

- **Primary:** home vegetable/flower gardeners with 2+ raised beds (the bed-map UI assumes this).
- **Judges (demo):** anyone who has ever killed a plant — i.e., everyone. The demo needs zero gardening expertise to enjoy.

3. Goals / Non-goals

Goals (MVP):

- An autonomous loop that maintains a small, current garden plan from fresh observations.
- A visible persist/discard boundary: archive everything, keep the working plan lean.
- A UI that makes the agent's memory *legible and beautiful* — the bed map is the memory.
- A 3-minute demo with two "wow" moments (self-rewriting plan, cross-season question).

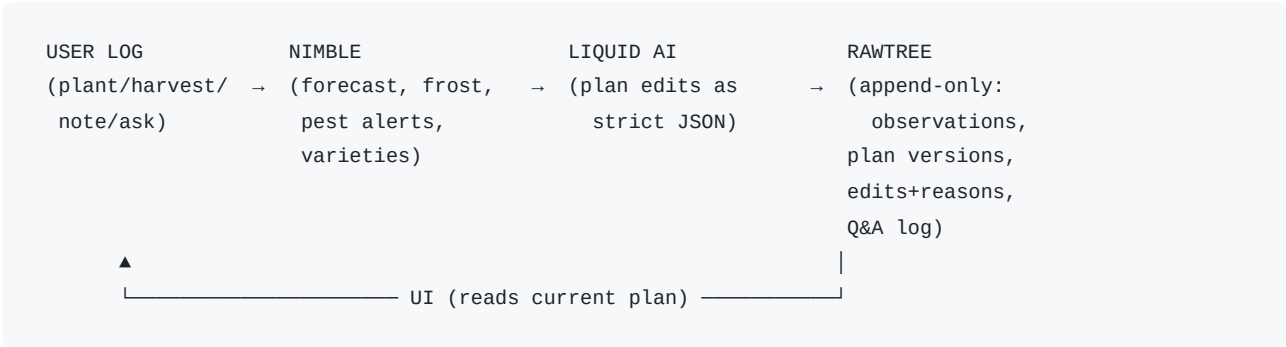
Non-goals (MVP): photo diagnosis, user accounts/multi-tenancy, mobile app, real-time push notifications, community features, payments. (Roadmap in README.)

4. User stories

1. As a gardener, I log a planting in 10 seconds so the garden's plan knows what's in the ground.
2. As a gardener, I see "this week" tasks that updated themselves when the forecast changed, so I don't have to re-plan.

3. As a gardener, I get warned about threats (frost, nearby pests) *before* they cost me a crop.
4. As a gardener, I ask "why did X fail last year?" and get an answer from my own history, not generic advice.
5. As a returning user, the garden remembers across seasons — variety performance per bed compounds yearly.

5. System architecture



The loop (`loop/main.py`), every N minutes or on demand:

1. Collect: user logs since last loop + Nimble observations (weather, frost risk, regional pest alerts).
2. Append raw observations to RawTree `observations` .
3. Call Liquid AI with: current plan + new observations + strict JSON schema. It returns plan edits.
4. Validate edits (schema + budget: plan must stay under ~600 words). Apply → new plan version in `plan_versions` ; each edit logged with reason in `plan_edits` .
5. Recompute derived views: this-week tasks, threat list, bed states.

Failure handling: malformed Liquid AI JSON → one retry with repair prompt → fall back to "no changes this loop" (never corrupt the plan). Nimble failure → loop continues on user logs alone. RawTree is append-only, so a bad loop never destroys history.

6. Data model (RawTree)

RawTree ingestion is append-oriented and schemaless — state is modeled as versioned events; "current" = latest version per entity.

Table	Shape	Notes
<code>observations</code>	<code>{ts, source, kind, content, loop}</code>	every Nimble pull + user log, raw

Table	Shape	Notes
plan_versions	{version, ts, loop, plan: {beds[], tasks[], threats[], notes}}	the mutable mind, snapshotted
plan_edits	{ts, loop, op, target, before, after, reason}	op ∈ KEEP/UPDATE/RETIRE/ADD; the audit trail
qa_log	{ts, question, answer, plan_version}	ask-the-garden history
loops	{loop, ts, duration_s, obs_count, edits_count, plan_words}	vitals

The plan object (bounded, ~600 words):

```
{
  "beds": [{ "id": "bed_1", "crop": "spinach", "stage": "seedling",
    "planted": "2026-03-02", "sun": "med", "water": "med", "note": "..."}],
  "tasks": [{ "id": "t_12", "title": "Cover peppers Thu night",
    "due": "2026-09-30", "priority": "high", "reason": "frost risk 2°C"}],
  "threats": [{ "kind": "pest", "title": "Aphids 2mi away", "status": "watching"}],
  "season_notes": ["Wet spring – reduced watering schedule"],
  "learnings": ["Bed 2 waterlogs – avoid carrots there"]
}
```

7. Tool integration details

Nimble — scheduled pulls, cached honestly as "last checked Xs ago":

- Local forecast (7-day, min temps → frost risk)
- Regional pest/disease alerts (extension services, gardening forums)
- Variety notes for logged cultivars
- Demo mode: scripted "cold snap Thursday" injection for the live rewrite moment

Liquid AI — two call types:

- *Gardener call* (frequent, cheap): system prompt = "you are the head gardener; given the current plan and new observations, return ONLY JSON edits per schema; keep the plan under 600 words; every edit needs a reason." Temperature low. Output validated before apply.
- *Answer call* (on demand): "answer using ONLY the current plan + relevant history; cite the plan version."

RawTree — via `rtree` CLI:

- `rtree insert --table observations --data '{...}'`
- `rtree query "SELECT * FROM plan_versions ORDER BY version DESC LIMIT 1"`
- Evolving columns accommodate new fields without migrations.

8. UI/UX spec

Design language: warm botanical editorial — cream paper, soil-brown ink, sage + terracotta accents, serif headings, botanical line illustration. Reference: `mockups/plot-plan-ui.webp`. This is the anti-dashboard: it should feel like a gardening magazine that thinks.

Panels (single page):

1. **Top bar** — wordmark "Perennial", season/week chip, LIVE pill, "+ Log planting" button.
2. **Garden Bed Map (hero)** — 6 raised beds top-down, each showing current crop at its growth stage; sun/water needs as small icons; beds pulse gently when their state changes. *This is the memory, visualized.*
3. **This Week** — task cards with priority chips; tasks appear/change/disappear as the plan rewrites. Each card shows its reason on hover ("why: frost risk 2°C Thu").
4. **Threat Watch** — calm alert cards; watching → active → resolved lifecycle.
5. **Ask the garden...** — free-text Q&A over current plan + history, with cited plan version.
6. **Season Timeline** — scrub Mar → Oct; past harvests as sprout markers; scrubbing shows the plan *as it was* (from `plan_versions`).
7. **Vitals strip** — moments logged, plan words (the leanness metric!), loops run, observations ingested.

Interaction principles: every state change animates (beds morph, tasks slide); every AI decision is inspectable (hover any task → the reason + the observation that caused it); nothing important is more than one click from its "why."

9. Retention mechanics (why users return)

- **Daily ritual:** the morning bed-walk becomes the morning check ("3 tasks, 1 threat").
- **Loss aversion:** one frost warning that saves a crop = a user for life.
- **Season arc:** plant → tend → harvest is a built-in progression loop.
- **Compounding:** year 2+ plans are built on your own results — the product gets better the longer you stay.

10. Demo script (3:00)

Time	Beat
0:00–0:30	"Agents die of memory obesity. Gardeners too." Show the living bed map.
0:30–1:30	Live rewrite: log "transplanted peppers, bed 5." Loop runs: Nimble sees Thursday's cold snap → Liquid AI edits the plan → "Cover peppers Thu night" appears. The plan changed itself.
1:30–2:15	Cross-season memory: "Ask the garden: why did my carrots fail last year?" → "Bed 2 stayed waterlogged — try bed 4's sandy soil." Then scrub the timeline back two years.
2:15–3:00	Reveal: 1,284 moments kept, plan still one page. "Explicit state, ruthless discard, total recall. That's the whole trick."

11. Success metrics (hackathon)

- Demo completes without manual intervention (autonomy).
- Plan stays under word budget across 20+ loops (the discard boundary holds).
- Both "wow" moments land (live rewrite + cross-season answer).
- Judges can describe the architecture back in one sentence ("it keeps a small plan and archives everything").

12. Risks & mitigations

Risk	Mitigation
Liquid AI returns malformed JSON	Schema validation + one repair retry; fallback = no-op loop
Nimble has no gardening API	Weather APIs + web search adapter; demo-mode scripted injection as backstop
RawTree access not provisioned	Abstract storage behind <code>rawtree_store.py</code> ; local JSONL fallback with identical interface
4-hour scope creep	MVP = loop + 6 panels + seed script. Photo diagnosis, accounts, and mobile are cut — listed as roadmap, not built.

13. Four-hour build plan

1. **Hour 1** — credentials + `rtree` working; one end-to-end loop in the terminal (log → observe → edit → store).

2. **Hour 2** — plan_edits reasons, Q&A path, seed script with 2 seasons; start the loop accumulating history.
3. **Hour 3** — UI panels bound to current plan; bed map SVG; timeline scrubber.
4. **Hour 4** — polish, failure-path testing (bad JSON, Nimble down), two full demo rehearsals, 30-min buffer.